

Concept Note: Features in Machine Learning

1 What are Features?

Features are the individual measurable properties or characteristics of the data that we use to train a machine learning model. They are the input variables that the algorithm uses to make predictions.

Think of features as the "clues" or "evidence" that help the machine make decisions.

1.1 Simple Analogy

Imagine you're buying a used car. What information do you consider?

- Mileage driven
- Year of manufacture
- Brand
- Engine capacity
- Number of previous owners
- Condition rating

In ML terms, each of these is a feature. Together, they help predict the car's price (the target/output).

2 Features vs. Target

Component	Description	Example (House Price Prediction)
Features (X)	Input variables used for prediction	Size, Location, Age, Bedrooms
Target (y)	Output variable we want to predict	House Price

Formula:

Features $\rightarrow$ [ML Model] $\rightarrow$ Target

$$X_1, X_2, X_3, \dots \rightarrow [\text{Algorithm}] \rightarrow y$$

3 Real-World Examples

3.1 Example 1: Email Spam Detection

Feature Name	Description	Example Value
word_count_free	Number of times "free" appears	3
exclamation_count	Number of exclamation marks	8
capital_ratio	Ratio of capital letters	0.35
email_length	Total characters in email	450
has_link	Does email contain links?	Yes (1)
sender_reputation	Trustworthiness score	2.5

Target: Spam (1) or Not Spam (0)

3.2 Example 2: Disease Prediction

Feature Name	Description	Example Value
age	Patient's age in years	45
blood_pressure	Systolic BP (mmHg)	140
cholesterol	Cholesterol level (mg/dL)	220
bmi	Body Mass Index	28.5
smoking	Smoker or not	Yes (1)
family_history	Family history of disease	Yes (1)

Target: Disease Present (1) or Absent (0)

3.3 Example 3: Student Performance Prediction

Feature Name	Description	Example Value
study_hours	Hours studied per week	15
attendance_rate	Class attendance percentage	85%
previous_grade	Last semester grade	78
sleep_hours	Average sleep per night	6.5
assignment_completion	Assignments completed (%)	90%

Target: Final exam score (0-100)

4 Types of Features

1. Numerical Features (Quantitative)

Values are numbers with meaningful magnitude.

- **Continuous:**

- Temperature: 23.5°C
- Weight: 65.2 kg
- Salary: \$55,000

- **Discrete:**

- Number of children: 2

- Age: 35 years
- Bedrooms: 3

2. Categorical Features (Qualitative)

Values represent categories or groups.

- **Nominal (no order):**

- Color: Red, Blue, Green
- Gender: Male, Female, Other
- City: New York, London, Tokyo

- **Ordinal (with order):**

- Education: *HighSchool* < *Bachelor's* < *Master's* < *PhD*
- Size: *Small* < *Medium* < *Large*
- Rating: *1star* < *2stars* < *3stars* < *4stars* < *5stars*

3. Binary Features

Only two possible values (often 0 or 1).

- **Has_insurance:** Yes (1) or No (0)
- **Is_smoker:** Yes (1) or No (0)
- **Email_contains_link:** True (1) or False (0)

4. Text Features

Derived from text data (requires special processing).

- **Original text:** "This product is amazing!"
- **Extracted features:**
 - **word_count:** 4
 - **sentiment_score:** 0.95 (positive)
 - **contains_exclamation:** 1
 - **word_amazing:** 1

5 Feature Engineering

Feature Engineering is the process of creating new features from existing data to improve model performance.

5.1 Example: Date Feature Engineering

- **Raw feature:**

- **date:** "2024-03-15"

- **Engineered features:**

- **year:** 2024
- **month:** 3 (March)

- day: 15
- day_of_week: 5 (Friday)
- is_weekend: 0 (No)
- quarter: 1 (Q1)
- is_holiday: 0 (No)

5.2 Example: Text Feature Engineering

- **Raw text:** "Machine Learning is awesome!!!"
- **Engineered features:**
 - text_length: 32
 - word_count: 4
 - avg_word_length: 6.5
 - capital_letter_count: 3
 - exclamation_count: 3
 - contains_positive_word: 1

6 Feature Representation in Code

How ML sees your data:

Original Data (Human-readable):

Size	Location	Age	Bedrooms	Price
1500	Downtown	5	3	\$300,000
2000	Suburb	10	4	\$400,000

Feature Matrix (ML-readable):

(Note: Categorical data like "Location" has been converted to numbers, e.g., Downtown=1, Suburb=0)

```
# X is the feature matrix (inputs)
X = [
    [1500, 1, 5, 3], # Sample 1
    [2000, 0, 10, 4] # Sample 2
]

# y is the target vector (outputs)
y = [300000, 400000]
```

7 Good Features vs. Bad Features

✓ Good Features:

- **Relevant:** Directly related to the target
- **Informative:** Provides useful signal
- **Non-redundant:** Not duplicating other features
- **Measurable:** Can be quantified or observed

× *Bad Features* :

- **Irrelevant:** No connection to target (e.g., eye color for house price)
- **Redundant:** Duplicate information (e.g., height in cm AND height in inches)
- **Constant:** Same value for all samples
- **Leaky:** Contains information from the future or target itself

8 Feature Selection

Not all features are useful. We need to select the best ones.

Example: Predicting Student Performance

All possible features:

- study_hours ✓ (Relevant)
- attendance_rate ✓ (Relevant)
- shoe_size × (*Irrelevant*)
- favorite_color × (*Irrelevant*)
- previous_grades ✓ (Relevant)
- student_id × (*Just an identifier, not predictive*)

Selected features:

study_hours, attendance_rate, previous_grades

9 The Role of Features in the ML Equation

Remember the equation from earlier? A simple linear model looks like this:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

Where:

y = Target (what we predict)

$x_1, x_2, x_3, \dots$ = Features (our inputs)

$\theta_1, \theta_2, \theta_3, \dots$ = Coefficients (what the model learns)

θ_0 = Bias/intercept

The algorithm learns which features are important by adjusting the θ (theta) values!

10 Key Takeaways

- Features are the input variables that describe each data point.
- Quality features lead to better models - "Garbage in, garbage out."
- Feature engineering can significantly improve performance.
- Different problems require different features.
- The same raw data can produce many different features through engineering.
- **In Machine Learning:** *Better features > Better algorithms*

A simple algorithm with great features often outperforms a complex algorithm with poor features!